

Legal Ease

Testing and Verifying Local Deployment

This guide walks through running Legal Ease on your machine and proving that document generation works, first with a manual smoke test, then with automated tests.

Note: the commands and tests below were written against the `routers.py` from the previous step. They were syntax-checked but not executed, because FastAPI was not available in the authoring environment. Run them locally and adjust module paths (such as `legal_ease.main:app`) to match your project.

1. Prerequisites

- Python 3.10 or newer (the code uses `str | None` syntax).
- The helper modules `routers.py` imports: `auth`, `db`, and `services` (`templates`, `analysis`, `renderer`, `storage`).
- `curl` for smoke tests; `poppler-utils` (`pdftotext`) is optional for inspecting output.

2. Set Up the Environment

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install fastapi "uvicorn[standard]" pytest httpx
```

3. Start the Server

```
uvicorn legal_ease.main:app --reload --port 8000
```

Then open `http://localhost:8000/docs`. If the interactive API page loads and lists the three `/documents` routes, the router is mounted correctly.

4. Manual Smoke Test

Replace `TOKEN` with a valid token for your local auth setup.

```
# 1. Request a document (expect HTTP 202 and a pending status)
curl -s -X POST localhost:8000/documents/generate \
  -H "Authorization: Bearer TOKEN" -H "Content-Type: application/json" \
  -d '{"doc_type":"nda","fields":{"party_a":"Acme","party_b":"Beta"}}'
```

```
# 2. Poll status using the returned id (expect "ready")
curl -s -H "Authorization: Bearer TOKEN" localhost:8000/documents/<id>
```

```
# 3. Download and inspect the PDF
curl -s -H "Authorization: Bearer TOKEN" localhost:8000/documents/<id>/download \
  -o out.pdf
file out.pdf && pdftotext out.pdf - | head
```

5. Automated Tests

The tests replace the database, template, renderer, and storage layers with small fakes, and override authentication. That isolates the router logic so the tests are fast and need no running server. Background tasks run before the test client returns, so a document is already ready when you check it.

```
pytest -v
```

test_routers.py

```
"""tests/test_routers.py - Local verification of the document generation router."""
from types import SimpleNamespace
```

```

import pytest
from fastapi import FastAPI
from fastapi.testclient import TestClient

from legal_ease import routers
from legal_ease.auth import get_current_user

class FakeRepo:
    def __init__(self):
        self.rows = {}

    def create(self, **kw):
        self.rows[kw["id"]] = dict(kw)
        return self.rows[kw["id"]]

    def get(self, id):
        return self.rows.get(id)

    def update(self, id, **kw):
        self.rows[id].update(kw)

REQUIRED = ("party_a", "party_b")
PAYLOAD = {"doc_type": "nda", "fields": {"party_a": "Acme", "party_b": "Beta"}}

@pytest.fixture
def client(monkeypatch, tmp_path):
    def save(doc_id, data):
        path = tmp_path / f"{doc_id}.pdf"
        path.write_bytes(data)
        return str(path)

    monkeypatch.setattr(routers, "documents_repo", FakeRepo())
    monkeypatch.setattr(routers, "templates", SimpleNamespace(
        render=lambda t, f: "contract text",
        missing_fields=lambda t, f: [k for k in REQUIRED if k not in f]))
    monkeypatch.setattr(routers, "analysis",
        SimpleNamespace(plain_summary=lambda text: "summary"))
    monkeypatch.setattr(routers, "renderer", SimpleNamespace(
        to_pdf=lambda text, summary=None: b"%PDF-1.4 test"))
    monkeypatch.setattr(routers, "storage", SimpleNamespace(save=save))

    app = FastAPI()
    app.include_router(routers.router)
    app.dependency_overrides[get_current_user] = lambda: SimpleNamespace(id="u1")
    return TestClient(app)

def test_generate_then_download(client):
    r = client.post("/documents/generate", json=PAYLOAD)
    assert r.status_code == 202
    doc_id = r.json()["id"]

    status = client.get(f"/documents/{doc_id}").json()
    assert status["status"] == "ready"
    assert status["download_url"] == f"/documents/{doc_id}/download"

    pdf = client.get(status["download_url"])
    assert pdf.status_code == 200
    assert pdf.headers["content-type"] == "application/pdf"
    assert pdf.content.startswith(b"%PDF")

def test_missing_fields_returns_422(client):
    r = client.post("/documents/generate",

```

```

        json={"doc_type": "nda", "fields": {"party_a": "Acme"}})
    assert r.status_code == 422
    assert r.json()["detail"]["missing_fields"] == ["party_b"]

def test_other_user_gets_404(client):
    doc_id = client.post("/documents/generate", json=PAYLOAD).json()["id"]
    client.app.dependency_overrides[get_current_user] = (
        lambda: SimpleNamespace(id="someone-else"))
    assert client.get(f"/documents/{doc_id}").status_code == 404

def test_failed_generation_blocks_download(client, monkeypatch):
    def boom(text, summary=None):
        raise RuntimeError("render failed")

    monkeypatch.setattr(routers, "renderer", SimpleNamespace(to_pdf=boom))
    doc_id = client.post("/documents/generate", json=PAYLOAD).json()["id"]

    assert client.get(f"/documents/{doc_id}").json()["status"] == "failed"
    assert client.get(f"/documents/{doc_id}/download").status_code == 409

```

6. Verification Checklist

Check	Expected result
Missing required field	422 with a missing_fields list
Valid request	202 Accepted, status pending, then ready
Download when ready	200, application/pdf, file starts with %PDF
Download when not ready or failed	409 Conflict
Another user's document id	404 Not Found (no leak of existence)
Renderer raises an error	Status becomes failed with an error message; API stays up
Interactive docs page	/docs loads and lists all three routes

7. Troubleshooting

- **ModuleNotFoundError: legal_ease:** run pytest and uvicorn from the project root, and make sure the package has an `__init__.py`.
- **401 on every request:** your token or auth dependency is not configured locally; use the dependency override shown in the tests.
- **Status stuck on pending:** the background task raised before updating the record, or the worker is not running if you moved to a queue.
- **Failed status with an error:** read the error field in the status response, then check the template name and renderer logs.
- **Port already in use:** start uvicorn with a different `--port` value.